Tutorial 2: Maximum Contiguous Subarray & Polynomial Multiplication
Exercise:

1. Follow the Lecture about the Maximum Contiguous Subarray to illustrate the MCS algorithm with the following input:

4	-3	5	-2	-1	2	6	-2

Ans.

$MCS(4,-3,5,-2,-1,2,6,-2) = MAX\{$
$MCS(4,-3,5,-2) = MAX\{$
$MCS(4,-3) = MAX\{$
$MCS(4) = (4)$
$MCS(-3) = (-3)$
$A = (4,-3)$
$\} = (4)$
$MCS(5,-2) = MAX\{$
$MCS(5) = (5)$
$MCS(-2) = (-2)$
$A = (5,-2)$
$\} = (5)$
$A = (4,-3,5)$
$\} = (4,-3,5)$
$MCS(-1,2,6,-2) = MAX\{$
$MCS(-1,2) = MAX\{$
$MCS(-1) = (-1)$
$MCS(2) = (2)$
$A = (-1,2)$
$\} = (2)$
$MCS(6,-2) = MAX\{$
$MCS(6) = (6)$
$MCS(-2) = (-2)$
$A = (6,-2)$
$\} = (6)$
$A = (2,6)$
$\} = (2,6)$
$A = (4,-3,5,-2,-1,2,6)$
$\} = (4,-3,5,-2,-1,2,6)$

2. Let A be an array of positive integers. Design a divide-and-conquer algorithm to compute the maximum value of $A[j] - A[i]$ with $j \geq i$. And analyze the time complexity of your algorithm.

   Ans.

   Intuition (high-level presentation):

   1) Split A into two subarrays $A[0, ..., mid]$ and $A[(mid + 1), ..., (n - 1)]$ from the middle.
   2) The position of $A[j]$ and $A[i]$ can be of three cases:
      a. $A[j]$ and $A[i]$ are both in $A[0, ..., mid]$
      b. $A[j]$ and $A[i]$ are both in $A[(mid + 1), ..., (n - 1)]$
      c. $A[j]$ in $A[(mid + 1), ..., (n - 1)]$ and $A[i]$ in $A[0, ..., mid]$
         i. ($A[j]$ in $A[0, ..., mid]$ and $A[i]$ in $A[(mid + 1), ..., (n - 1)]$ is impossible because $j \geq i$)
   3) For case a and b, we go to recursions.
   4) For case c, $A[j]$ is the largest value in $A[(mid + 1), ..., (n - 1)]$ and $A[i]$ is the smallest value in $A[0, ..., mid]$.
   5) The recursion terminates when A has one element. The algorithm returns 0.

Algorithm 1: $MaxDiff(A)$

Input: an array A of $n > 0$ integers

Output: the maximum of $A[j] - A[i]$, $0 \leq i \leq j \leq n - 1$

```
1.  begin
2.    if n = 1 then
3.      return 0;
4.    else
5.      mid = ⌊(n−1)/2⌋
6.      DiffLeft = MaxDiff(A[0, ..., mid])
7.      DiffRight = MaxDiff(A[(mid + 1), ..., (n − 1)])
8.      min = A[0]
9.      for x = 0 to mid do
10.       if min > A[x] then
11.         min = A[x]
12.       end if
13.     end for
14.     max = A[(mid + 1)]
15.     for x = mid + 1 to n − 1 do
16.       if max < A[x] then
17.         max = A[x]
18.       end if
19.     end for
20.     return max{DiffLeft, DiffRight, max − min}
21.   end if
22. end
```

Time complexity analysis:

(1) Obviously, when $n = 1$, $T(n) = 2$

(2) When $n > 1$

Algorithm 1: $MaxDiff(A)$

Input: an array A of $n > 0$ integers

Output: the maximum of $A[j] - A[i]$, $0 \le i \le j \le n-1$

```
1.  begin
2.    if n = 1 then                                          // O(1)
3.       return 0;
4.    else
5.       mid = ⌊(n−1)/2⌋                                     // O(1)
6.       DiffLeft = MaxDiff(A[0, ..., mid])                  // T(n/2)
7.       DiffRight = MaxDiff(A[(mid + 1), ..., (n − 1)])     // T(n/2)
8.       min = A[0]                                          // O(1)
9.       for x = 0 to mid do                                 // O(n)
10.         if min > A[x] then                                   // O(1)
11.            min = A[x]                                       // O(1)
12.         end if
13.      end for
14.      max = A[(mid + 1)]                                  // O(1)
15.      for x = mid + 1 to n − 1 do                         // O(n)
16.         if max < A[x] then                                   // O(1)
17.            max = A[x]                                       // O(1)
18.         end if
19.      end for
20.      return max{DiffLeft, DiffRight, max − min}          // O(1)
21.   end if
22. end
```

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \qquad \text{if } n \ge 2$$

Solve the recurrence relation

$$T(n) \le 2T\left(\frac{n}{2}\right) + cn$$

$$\le 2[2T\left(\frac{n}{2^2}\right) + c\frac{n}{2}] + cn$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + 2cn$$

$$\le 2^2[2T\left(\frac{n}{2^3}\right) + c\frac{n}{2^2}] + 2cn$$

$$= 2^3 T\left(\frac{n}{2^3}\right) + 3cn$$

$$\vdots$$

$$= 2^i T\left(\frac{n}{2^i}\right) + icn$$

Let $i = \log n$.

$T(n) \leq nT(1) + \log n \cdot cn = O(n \log n)$

3. Consider two complex number $a + bi$ and $c + di$ where $i = \sqrt{-1}$.
How to compute their product with only 3 numerical multiplications? (Note that there is no requirement on the number of additions and subtractions.)

Ans.

$$\begin{aligned} & (a + bi) \times (c + di) \\ = \; & ac + (ad + bc)i - bd \\ = \; & ac + (ac + ad + bc + bd - ac - bd)i - bd \\ = \; & ac + \big((a + b)(c + d) - ac - bd\big)i - bd \end{aligned}$$

Thus, we only need to compute ac, bd, and $(a + b)(c + d)$, which are three multiplications.